실시간 에지 검출 파이프라인

OpenCV · 실습 · 웹캠 · Canny

cchyun@gmail.com

Part 1. 파이프라인 전체 구조

실시간 에지 검출 파이프라인이란?

- **원본 영상에서 의미 있는 정보(경계선)를 추출하기 위한 단계별 처리 흐름**

  - **대상:** Python과 OpenCV를 처음 접하는 학생 대상 실습
  - **목적:** 로봇의 '시각'에 해당하는 컴퓨터 비전의 기초를 직접 구현

  - **로봇 시각 시스템의 역할**
    - 광학 센서, 영상 처리 소프트웨어, 로봇 제어 로직을 통합
    - 사물을 감지하고 공간 좌표를 결정

파이프라인 5단계 구성

- **파이프라인은 5단계로 구성됩니다**

1. **영상 입력 (Input)**
   - 웹캠을 통해 실시간 프레임을 읽거나 이미지 파일을 불러옴

2. **색공간 변환 (Grayscale Conversion)**
   - 3채널 컬러(BGR) → 1채널 흑백 영상
   - 데이터 복잡성을 줄이고 처리 속도를 높이며 구조적 정보는 유지

3. **가우시안 블러 (Gaussian Blur)**
   - 노이즈를 제거하여 영상을 부드럽게 만듦
   - 에지 검출기는 급격한 픽셀 변화에 민감 → 노이즈 먼저 제거 필수

파이프라인 5단계 구성 (계속)

- **에지 검출 및 결과 표시**

4. **Canny 에지 검출 (Canny Edge Detection)**
- 미세한 질감 변화보다 사물의 **윤곽선**을 정확하게 추출하는 다단계 알고리즘

5. **결과 표시 (Display)**
- 가공된 프레임을 실시간으로 화면에 출력

```
[웹캠 입력] → [Grayscale 변환] → [가우시안 블러] → [Canny 검출] → [화면 출력]
```

Part 2. 단계별 구현 전략

점진적 구현 방식

- **한 번에 모든 기능을 넣기보다, 기능을 하나씩 추가하며 작동을 확인**
  - **Step 1: 웹캠 영상 띄우기**
    - 원본 영상이 화면에 끊김 없이 나오는지 먼저 확인
  - **Step 2: 전처리 적용**
    - 흑백 변환과 블러 처리를 순차적으로 추가하여 영상 변화 확인
  - **Step 3: 에지 검출 적용**
    - 최종적으로 Canny 알고리즘을 파이프라인 끝에 연결
  - **Step 4: 심화 미션 수행**
    - 파라미터 조절이나 마스킹 기능을 추가

Part 3. 완성 코드 구현

실시간 에지 검출기 — 전체 코드 (1/2)

```python
import cv2        # OpenCV 라이브러리 임포트
import numpy as np  # 행렬 연산을 위한 NumPy

def edge_detection_pipeline():
    # 1. 웹캠 연결 (0번은 내장 웹캠)
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("카메라를 열 수 없습니다.")
        return

    while True:
        # 프레임 읽기
        ret, frame = cap.read()
        if not ret:
            break

        # 2. 색공간 변환: BGR -> Gray
        # 에지 검출 전 복잡도를 낮추기 위해 수행
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

실시간 에지 검출기 — 전체 코드 (2/2)

```
        # 3. 가우시안 블러: 노이즈 제거
        # (5, 5)는 커널 크기 — 숫자가 클수록 더 많이 흐려짐
        blurred = cv2.GaussianBlur(gray, (5, 5), 0)

        # 4. Canny 에지 검출
        # threshold1=100, threshold2=200 설정
        edges = cv2.Canny(blurred, 100, 200)

        # 5. 결과 표시
        cv2.imshow('Original Video', frame)
        cv2.imshow('Edge Detection Pipeline', edges)

        # 'q' 키를 누르면 종료 (25ms 대기)
        if cv2.waitKey(25) & 0xFF == ord('q'):
            break

    # 리소스 해제
    cap.release()
    cv2.destroyAllWindows()

if __name__ == "__main__":
    edge_detection_pipeline()
```

코드 핵심 포인트

- **각 단계별 핵심 API 정리**

단계	함수	주요 파라미터
웹캠 연결	`cv2.VideoCapture(0)`	`0` = 내장 웹캠
색공간 변환	`cv2.cvtColor()`	`cv2.COLOR_BGR2GRAY`
가우시안 블러	`cv2.GaussianBlur()`	커널 크기 `(5, 5)`
Canny 검출	`cv2.Canny()`	`threshold1=100, threshold2=200`
결과 표시	`cv2.imshow()`	창 이름, 프레임
키 대기	`cv2.waitKey(25)`	대기 시간(ms)

Part 4. 웹캠 없을 때 대체 방법

이미지 파일로 실습하기

- **카메라가 없는 환경이라면** `cv2.imread()` **로 정적 이미지 파일을 사용**

```
# 웹캠 대신 이미지 파일 읽기
img = cv2.imread('test_image.jpg')

if img is None:
    print("이미지 파일을 읽을 수 없습니다.")
else:
    # 파이프라인의 영상 처리 부분을 그대로 적용
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(blurred, 100, 200)

    cv2.imshow('Result', edges)
    cv2.waitKey(0)  # 아무 키나 누를 때까지 창 유지
```

- **주의:** `cv2.waitKey(0)` → 정적 이미지는 키 입력 전까지 창 유지
- **차이:** 실시간 영상은 `waitKey(25)`, 정적 이미지는 `waitKey(0)`

Part 5. 자주 발생하는 오류와 해결법

오류 1 & 2 — Assertion Failed / 창이 닫힘

- **오류 1:** `cv2.error: (-215:Assertion failed) ...`
  - **원인:** `cv2.imread()`나 `cap.read()`가 이미지를 제대로 읽지 못해 변수가 `None`인 경우
  - **해결:**
    - 파일 경로가 올바른지 확인
    - 카메라 접근 권한(macOS 개인 정보 보호 설정)이 허용되어 있는지 확인

- **오류 2: 창이 바로 닫히거나 응답 없음**
  - **원인:** `cv2.waitKey()`가 루프 안에 없거나 대기 시간이 0인 경우
  - **해결:** 실시간 영상에서는 반드시 `waitKey(1)` 또는 `waitKey(25)` 사용
    - 시스템이 이벤트를 처리할 시간을 주어야 함

오류 3 & 4 — 색상 이상 / 에지 검출 불량

- **오류 3: 영상 색깔이 이상함**
  - **원인:** OpenCV는 **BGR** 순서로 처리하지만, Matplotlib 등은 **RGB** 순서 사용
  - **해결:** 시각화 라이브러리에 맞춰 `cv2.cvtColor()`로 순서를 변환

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

- **오류 4: 에지 검출 결과가 너무 복잡하거나 선이 안 보임**
  - **원인:** Canny의 임계값(Threshold)이 현재 환경(조명 등)에 맞지 않기 때문
  - **해결 방법:**
    - 가우시안 블러 커널 크기를 키워 노이즈 억제 (`(7, 7)` 등)
    - `threshold1`, `threshold2` 값을 환경에 맞게 조절

Part 6. 심화 미션

미션 1 — 트랙바로 Canny threshold 실시간 조절

- `cv2.createTrackbar`를 사용하여 Canny 임계값을 마우스로 실시간 조절
  - **목표:** 영상 재생 중 두 가지 임계값을 조절하여 최적값 탐색
  - **의미:** 로봇이 다양한 조명 환경에서 최적의 파라미터를 찾는 데 필수적인 과정

```
cv2.namedWindow('Controls')
cv2.createTrackbar('Low Threshold',  'Controls', 100, 255, lambda x: None)
cv2.createTrackbar('High Threshold', 'Controls', 200, 255, lambda x: None)

# 루프 내부
low  = cv2.getTrackbarPos('Low Threshold',  'Controls')
high = cv2.getTrackbarPos('High Threshold', 'Controls')
edges = cv2.Canny(blurred, low, high)
```

미션 2 — 색상 마스킹 + 에지 검출 결합

- **특정 색상 영역(예: 파란색)만 추출하는 마스킹 기법을 에지 검출 앞에 추가**
  - **목표:** 배경 노이즈는 무시하고, '파란색 상자'의 윤곽선만 추출하는 파이프라인 구축

```
# BGR → HSV 변환
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

# 파란색 범위 정의
lower_blue = np.array([100, 150, 50])
upper_blue = np.array([140, 255, 255])

# 마스크 생성 및 적용
mask = cv2.inRange(hsv, lower_blue, upper_blue)
masked = cv2.bitwise_and(gray, gray, mask=mask)

# 마스킹 된 영역에만 에지 검출 적용
edges = cv2.Canny(masked, 100, 200)
```

미션 3 — 에지 검출 결과를 파일로 저장

- **중요한 프레임이나 에지 검출 결과 영상을 파일로 저장**

  - **방법**: 루프 중간에 특정 키(`'s'`)를 누르면 `cv2.imwrite()`가 실행되도록 코드 추가
  - **활용**: 저장된 데이터는 로봇의 학습 데이터나 보고서용 자료로 활용

```
key = cv2.waitKey(25) & 0xFF

if key == ord('s'):  # 's' 키: 현재 프레임 저장
    cv2.imwrite('result.png', edges)
    print("에지 이미지 저장 완료: result.png")

if key == ord('q'):  # 'q' 키: 종료
    break
```

  - **키 처리 정리**: `waitKey()` 반환값을 변수에 저장하면 여러 키를 동시에 처리 가능